MCNN*: A mixed convolutional approach for 3D object classification

Noyan Erdin Kilic† Ahmed Hisham Abdelghaffar Aboulenien† Michele Zarantonello†

***Abstract*—Three-dimensional object recognition is one of the most crucial challenges in the computer vision and robotics field. Good results were achieved using CNNs and orientation information,; however, many datasets do not have orientation information and coming up with it is usually a challenge on its own. We propose an improvement over the original VoxNet paper without needing the orientation data by training the network not just on the 3D voxel grid, but also by training 2D CNNs on the different 2D projections of the object. This approach, combined with some other minor tweaks, leads to an accuracy of 86.7% on the ModelNet40 test set. The main draw of this method is that it is simple to implement and fast to train on relatively weak GPUs (training time $<$ 10 mins). It is also generalizable to almost any other 3D object dataset as it requires minimal preprocessing or additional information. The code for our model is available on *https://github.com/nyanklc/3d_classifier*.**

***Index Terms*—Supervised Learning, 3D Classification, Voxel Grid, Convolutional Neural Networks.**

I. Introduction

Three-dimensional object recognition has become a very in-demand task in many applications. With access to advanced sensors becoming easier [4], more and more robots are now including them to aid in recognizing and navigating their environment. In addition, they have proved very helpful for autonomous cars. This creates a great need for simple yet effective networks that are able to classify a wide variety of objects.

One of the proposed architectures in the literature is to perform voxelization on the object and then use a CNN to try to classify it [1]. A voxel (volumetric pixel) is a 3D pixel that contains information on the object. In our case it is a simple binary digit representing whether this area is solid or empty, The original VoxNet architecture achieved a performance of $\mathbf{83}\%$ on the ModelNet40 dataset. While ORION was able to achieve over $\mathbf{89}\%$ [2] their method requires orientation data which is a task on its own and required either manual labeling or training a separate network and even then it was not a straightforward process and not easily generalizable (for example, it required different rules for different objects).

In this work we propose a middle ground solution that has higher accuracy than VoxNet but still does not require extra orientation data. Our approach relies on using information that already exists in the voxel grid but not well exploited, which are the 2D features of the object from different views (top,side..etc). We achieve this by doing a simple projection onto different axis. The features extracted are then concatenated with the ones extracted by the 3D CNN. This is in a way similar to what humans do when they see an unrecognized object, they try to look at it from different perspectives to better understand it. This is very cheap computationally as the projection is a simple max operation along the required axis. Furthermore, The 2D CNN is small compared to the 3D one making its impact on the performance marginal. This method also proved to be generalizable to all the object classes without any problems so it can be safely used without a lot of considerations.

In addition, we added a third convolutional layer in the 3D CNN, added BatchNorm and Leaky-ReLU layers, used ADAM optimizer and an adaptive learning rate that halves if validation accuracy does not increase for 3 epochs. These minor hyper-parameter adjustments helped to further increase the performance.

This report is structured as follows: In Sec. II we describe closely related previous work done in this field/task. The processing done on the data are explained in Sec. III. The methodology and detailed model architecture are presented in Sec. IV, and the model's performance evaluation is carried out in Sec. V. Concluding remarks are provided in Sec. VI.

II. Related Work

Early work on tasks that involve learning from 3D objects explored voxelized representations of shapes. Wu *et al.* introduced **3D ShapeNets** [3], employing a convolutional deep belief network to learn probabilistic voxel occupancy grids. They have constructed a dataset consisting of CAD models known as **ModelNet** to train their model, and they have become one of the first to build a 3D deep learning model. Building on this foundation, Maturana and Scherer proposed **VoxNet** [1], the first convolutional neural network explicitly designed for 3D voxel grids. VoxNet showed that lightweight 3D CNNs could achieve real-time performance on datasets such as ModelNet. Even though they have achieved state of the art performance with their work when they first published it, newer methods tackling the same task have superseded their performance. An example of these would be **ORION** [2] introduced by Sedaghat *et al.*, an orientation-aware extension of voxel CNNs. ORION makes use of an auxiliary task during the training of their model, by using orientation labeling on the datasets and forcing their model to also learn to predict the orientation of the input, even though this information is discarded for inference on the classification task. The work

* M stands for MONSTER: Multi-dimensional vOxel Network for Spatial feaTure ExtRaction

noyanerdin.kilic@studenti.unipd.it
ahmedhishamabdelghaffar.aboulenien@studenti.unipd.it
michele.zarantonello@studenti.unipd.it

presented by us does not take a similar approach, meaning no additional information such as orientation labels are used. Instead, only the raw voxelized input data is used for the classification task.

Instead of using 3D convolutional networks, some approaches utilize 2D views of the input object. **RotationNet** [5] was trained to classify objects along with estimating their pose by using information not from the 3D data, but from multiple 2D views of the model, however their approach relied on predetermined viewpoints.

In our approach, we build upon prior work by integrating voxel-based 3D CNNs with a complementary 2D view strategy. Specifically, we employ a VoxNet-inspired 3D feature extractor alongside a 2D feature extractor, and concatenate their outputs before feeding them into a fully connected classifier. Experimental results demonstrate that incorporating 2D views of the voxelized input enhances prediction accuracy compared to relying solely on 3D CNN features.

III. Data/Input Processing

A. Preprocessing and Voxelization

To classify objects, a dictionary is used to assign each label a corresponding index. After that, some minor file adjustments are made to make them compatible with the Open3D library that we use to import the meshes. We write a function which ensures that the first line has to contain only "OFF" and the mesh coordinates start from the second line; otherwise, it modifies the file to that.

The ModelNet40 dataset provides CAD models in mesh format (.off). Since meshes can vary in scale and position, we first normalize each model by centering it within its axis-aligned bounding box and scaling it to fit inside a unit cube. This ensures that all objects, regardless of their original dimensions, are consistently represented in the voxel grid. After normalization, each mesh is voxelized using the Open3D library. The voxelization converts the triangular mesh into a binary occupancy grid, where each element of the grid encodes whether the corresponding spatial region is occupied by the object or not. We investigated different grid resolutions and we came up with 33x33x33 as the best model grid shape. We tried also higher resolutions (such as 51x51x51) but both the performance and the computational efficiency were worse.

B. Data Augmentation and Voting

The ModelNet40 dataset is not pre-aligned, and due to struggles in implementing the alignment of the data, we came up with other solutions to make the learning robust to rotations because our representation has no built-in invariance to large rotations. First we tried with offline rotations. We augmented the dataset by generating rotated versions of each object at fixed angles around the vertical axis: each augmented model is stored as a new training sample. This increases dataset size by a factor of eight and provides the network with exposure to multiple canonical orientations. However, it did not bring the expected improvement, most likely because generating the same object rotated 8 times (45°, 90°, 135°, 180°, 225°, 270°, 315°) makes the model overfit. We gave up with the idea of augmenting the dataset offline and deterministically. To solve this problem we implemented online random rotations during training. Unlike the deterministic offline approach, this ensures that the model sees a different orientation at every epoch, reducing the risk of overfitting to repeated copies of the same rotations.

For testing and inference, a voting system [2] was implemented so that the model output is acquired for multiple rotated versions of the input, and these different outputs were fused to determine the final class prediction. These methods were not kept in the final implementation of the model, since their contribution were not found to be significant.

The ModelNet40 dataset is already split into training and testing. For the validation dataset, before the training started, we would randomly split our data into training and validation. We also scripted the splitting in a way to ensure that, in case of training with an augmented dataset, if an object is included in training, all its rotations is present with it (validation dataset was changed accordingly). This was done to prevent data leaking from training to validation.

IV. The Model Architecture

A. MCNN Architecture

1) High level overview: Our main approach to the classification task consists of feature extraction blocks that process the voxelized input data, and a feedforward classifier that converts the extracted features into the classification output. A high level description of the architecture can be seen in Fig. 1.

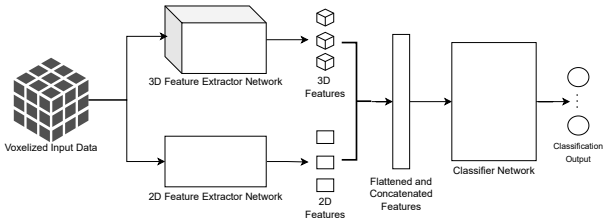


Fig. 1: High level overview of the architecture.

In the VoxNet [1] architecture, a similar architecture is present with a 3D feature extractor. However, in our model, the main motivation of adding another feature extractor with a lower dimension was to be able to provide more information about the object to the classification network.

2) Detailed description: We implemented a hybrid architecture that integrates a 3D convolutional feature extractor with a 2D convolutional feature extractor applied to different projected views of the voxel grid. The rationale behind this design is to leverage both local volumetric structures (captured by 3D convolutions) and global shape contours (captured by 2D projections).

The 3D convolutional feature extractor coincides with the convolutional layers of our improved VoxNet (each layer followed by batch normalization and LeakyReLU activations

and a max pooling layer with reduction factor 2 before the third layer). The 2D convolutional feature extractor shares the same implementation.

To capture the images of the object along the three principal axes, we compute three projections of the voxel occupancy grid by applying a maximum operation on each axis. These projections reduce the 3D volume into 2D occupancy maps that summarize object contours and exterior boundaries.

The final feature representation is obtained by concatenating the flattened 3D feature vector with the three 2D projection feature vectors. The concatenated vector has dimensionality 25,600 before being passed to the classifier.

The classifier is composed of a fully connected layer that reduces the dimensionality to 128 units, a dropout layer, a LeakyReLU activation and a final linear layer with 40 outputs. The output activations are not present since the implementation of the loss function used (cross entropy) already applies a LogSoftMax operation during calculations.

Fig 2 demonstrates in detail the MCNN architecture.

B. Alternative Model Architectures

Starting from the baseline VoxNet, we implemented and tested several architectural variations to investigate how different design choices affect classification performance.

1) Improved VoxNet: Input Layer The network accepts as input a fixed-size voxel occupancy grid of IxJxK. Same as VoxNet, we set I=J=K=33.

Convolutional Layers The feature extraction stage consists of three 3D convolutional layers. The first layer applies f=40 filters of size 3x3x3 with stride s=2, reducing so the dimension of the input.

The second layer increases the feature depth to 80 with a kernel size of 3 and stride 1, finally, a third convolutional layer outputs 128 feature maps, again using a kernel size of 3 and stride 1. After each convolution, batch normalization is applied to stabilize learning and prevent co-adaptation, followed by a LeakyReLU nonlinearity to preserve gradient flow even for negative activations.

Batch normalization allows each layer to learn independently of the scale of its inputs, mitigating covariate shift, while LeakyReLU helps prevent dead neurons by allowing small negative gradients.

Pooling layer A single 3D max-pooling layer is placed after the second convolutional block reducing the feature maps dimensionality by a factor of 2. This reduces the spatial size of the features from 14x14x14 to 7x7x7.

Fully Connected Layers The flattened output of the final convolutional block produces a vector of dimension 16,000, which is projected into a latent space of 128 dimensions via a fully connected layer. A dropout layer is included at this stage to mitigate overfitting, followed by a LeakyReLU activation (VoxNet used ReLU). The final classification layer is a fully connected layer mapping to 40 outputs, corresponding to the object categories in ModelNet40.

The improved VoxNet architecture contains 2,417,992 parameters, and can be summarized as:

C(40, 3, 2) – BN – LReLU – C(80, 3, 1) – BN – LReLU – P(2) – C(128, 3, 1) – BN – LReLU – FC(128) – Dropout – LReLU – FC(40)

2) Autoencoder+Classifier: We trained a 3D convolutional autoencoder to reconstruct voxelized objects. Once the autoencoder achieved a satisfactory accuracy, the decoder was discarded, and the encoder weights were saved to be used later. The encoder produces a compact embedding of the voxel grid, which can then be fed to the usual linear classifier.

The encoder convolutional network composes of three convolutional layers. LeakyReLU nonlinearity and batch normalization are adopted. Stride and max-pooling are used to apply dimensionality reduction. In our implementation the encoder configuration is:

- Conv3D (in=1, out=4, kernel=3, stride=2) - LeakyReLU - BatchNorm3D(4)
- Conv3D (in=4, out=8, kernel=3, stride=1) - LeakyReLU - MaxPool3D(2)
- Conv3D (in=8, out=16, kernel=3, stride=2) - BatchNorm3D(16) - LeakyReLU

This encoder compresses the voxel input into a small volumetric representation which is then flattened to yield a latent vector. We also experimented new approach to improve this architecture. In particular, we tried the idea of concatenating these embeddings with features extracted from the improved VoxNet, and with features extracted from auxiliary 2D CNNs.

3) Variants: Input inversion Besides the standard voxel occupancy grids, we also explored their binary complements, where zeros and ones are swapped. The intuition is that the inverted representation emphasizes object contours and voids. We extended the architecture to process both normal and inverted inputs, and concatenated the extracted features before classification. This forces the network to learn representations that capture both the solid structure and the surrounding empty space.

Axis Permutations We experimented with permuting the axes of the voxel grid. For each permutation, features were extracted by the convolutional backbone, and the final descriptor was obtained by pooling across permutations. This technique is designed to reduce orientation sensitivity and encourage invariance to axis ordering, though in practice we found it more computationally demanding.

V. Results

The performance of various discussed architectures, along with our main model MCNN can be examined in Tab. 1.

The classification accuracy of the MCNN model (and MCNN model with different hyperparameters for comparison) during training can be seen in Fig. 8. The ”vanilla” MCNN model outperforms others by a small margin.

The training and validation losses for these models can be seen in Fig 5. The MCNN model with its selection of hyperparameters achieves a lower validation loss throughout training than others.

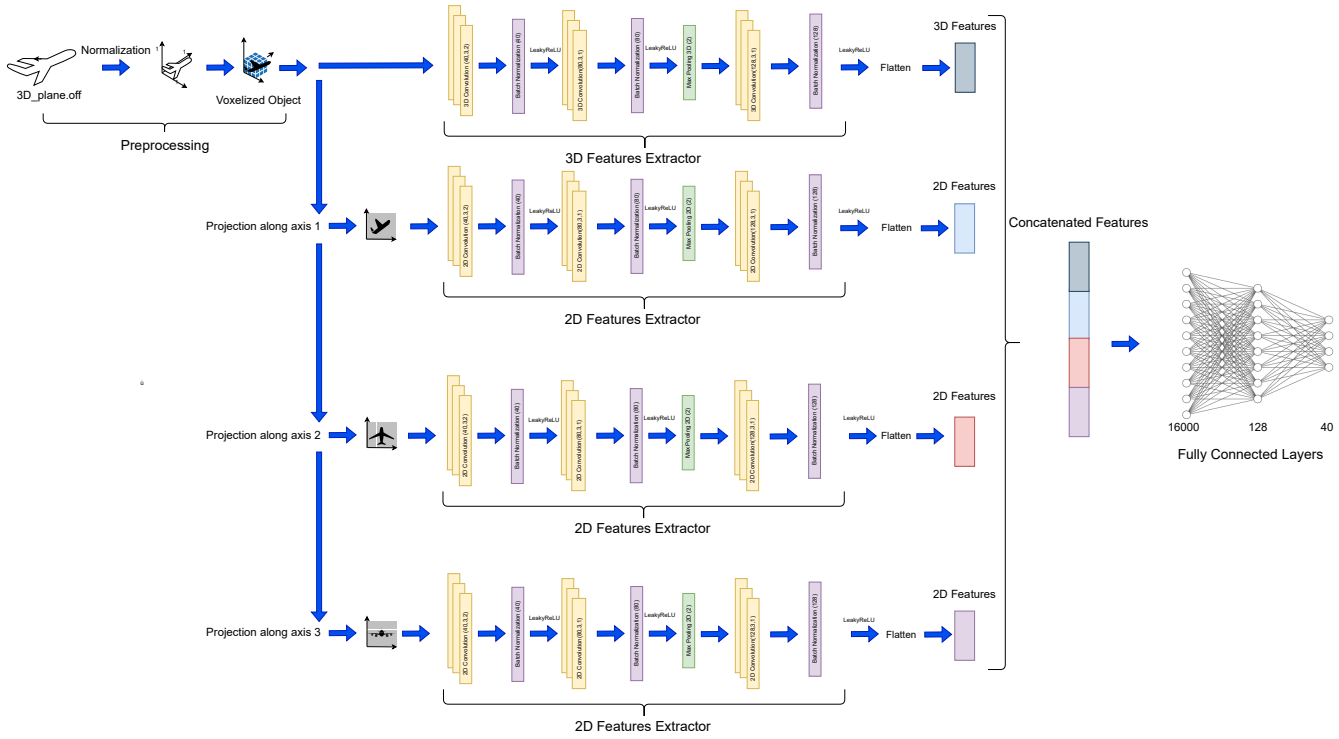


Fig. 2: Architecture of the MCNN model.

TABLE 1: MCNN ablation results on ModelNet40.

Method	**Accuracy (%)**
MCNN with offline rotation augmentations + voting	77.6
MCNN with online rotation augmentations + voting	78.9
Autoencoder	79.9
VoxNet	83.0
MCNN with SGD	85.3
MCNN without 2D layers	86.0
MCNN without LR scheduler	85.4
MCNN with 51 voxel grid size	86.4
MCNN	**86.7**
ORION	89.7

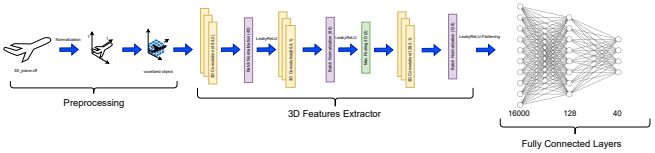


Fig. 3: Improved VoxNet architecture.

The confusion matrix of the classification results, both on the last validation pass during training and the test dataset, can be seen in Figs. 6 and 7.

Looking at the results, a few observations can be made:

- Validation is the highest for MCNN over multiple runs, and it shows that the selection of hyperparameters for it are more optimal compared to others. The final test accuracy is also higher compared to its variants.
- One major success of MCNN is that it keeps the training loss/accuracy at a lower rate, while having a higher validation score. This emphasizes that adding a 2D convolutional network along with a 3D feature extractor helped MCNN be less prone to overfitting. Usage of a learning rate scheduler with an already existing ADAM optimizer was beneficial for this.
- The missclassified inputs, understood by examining the confusion matrices, shows that the model makes mistakes (gets confused) with inputs that have similar 3D structure. To overcome this issue, more emphasis on certain input classes can be made via augmentation, or a separate decisive model can be implemented specifically for those

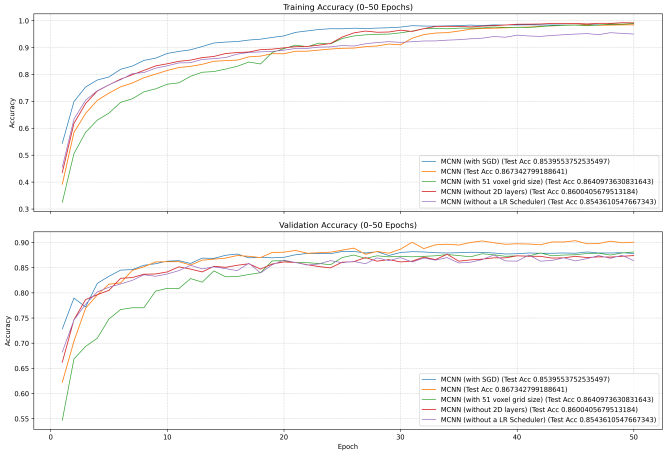
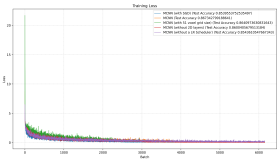
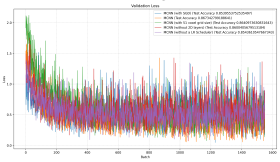


Fig. 4: Training/Validation Accuracies

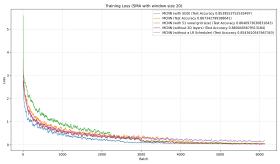
classes.



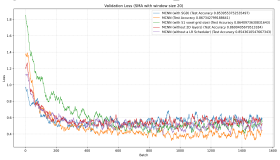
(a) Training Loss



(b) Validation Loss



(c) Training Loss with Simple Moving Average



(d) Validation Loss with Simple Moving Average

Fig. 5: Training/Validation Loss Plots of MCNN

VI. Concluding Remarks

The MCNN model architecture was introduced, and its performance was evaluated. Based on VoxNet architecture, and with a new addition of a 2D feature extractor, it has been shown that performance on the ModelNet40 dataset was improved.

We learned that data augmentation does not always lead to better results and can sometimes cause overfitting. One of

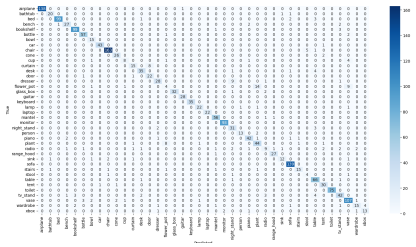


Fig. 6: Confusion Matrix (Validation)

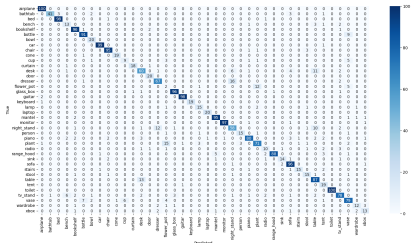


Fig. 7: Confusion Matrix (Test)

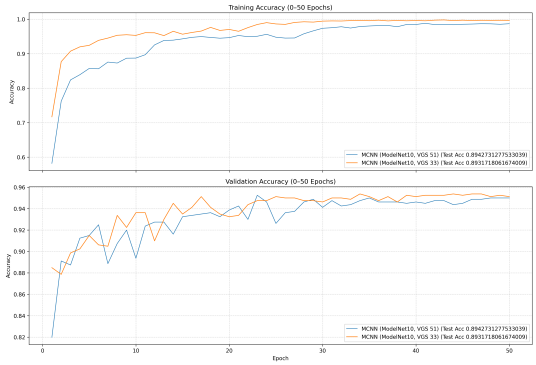


Fig. 8: Training/Validation Accuracies

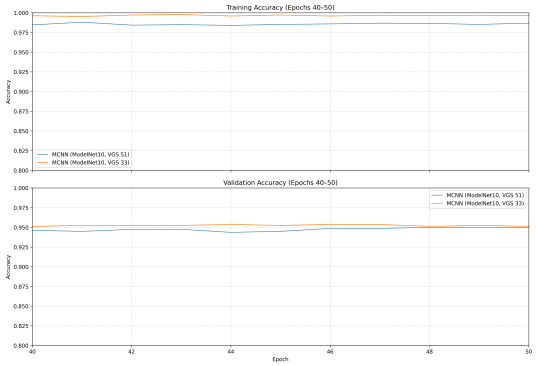


Fig. 9: Training/Validation Accuracies (Zoomed)

the problems that we faced was trying different things that theoretically should improve the performance, but they did not. We've seen that focusing on hyperparameter optimization is as important as coming up with a better model structure/architecture, and it can be very important in achieving a better

performance. We have tried lots of different structures to come up with a better result, and we have also seen a significant improvement from optimizing the training phase.

REFERENCES

[1] D. Maturana and S. Scherer, "Voxnet: A 3d convolutional neural network for real-time object recognition," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 922–928, 2015.

[2] N. Sedaghat, M. Zolfaghari, E. Amiri, and T. Brox, "Orientation-boosted voxel nets for 3d object recognition," in *Proceedings of the British Machine Vision Conference (BMVC)*, pp. 1–13, 2016.

[3] Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, and J. Xiao, "3d shapenets: A deep representation for volumetric shapes," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1912–1920, 2015.

[4] D. Liang *et al.*, "Evolution of laser technology for automotive lidar, an industrial viewpoint," *Nature Communications*, vol. 15, no. 1, pp. 1–11, 2024.

[5] A. Kanezaki, Y. Matsushita, and Y. Nishida, "Rotationnet: Joint object categorization and pose estimation using multiviews from unsupervised viewpoints," 2018.